

# DSBA 6165: Final Project Report

## Super Resolution for Video Games

Eric Phann, Samantha Michael, Kidus Kidane

[Github Link](#)

## Introduction

Modern video games are becoming increasingly graphically intensive, forcing gamers to choose between high visual quality and smooth performance. To strike this balance, developers use upscaling techniques like NVIDIA's Deep Learning Super Sampling (DLSS), which renders frames at lower resolutions and upscales them using deep learning. But since DLSS is proprietary, there's growing interest in open-source alternatives.

Our goal is to make high quality upscaling more accessible, helping users with mid range hardware enjoy demanding games. We use a machine learning based approach to Image Super Resolution (ISR) that enhances visual detail without heavy computational costs, addressing both gamer needs and computer vision challenges.

ISR has evolved from early CNNs like SRCNN to deeper models like VDSR. SRGAN and ESRGAN brought in adversarial training, and transformers now represent the state of the art in visual tasks. While these advancements have shown impressive results on standard benchmarks and photography datasets, their application to video game content presents unique challenges due to the synthetic nature of game graphics, diverse art styles, and real-time performance requirements.

This gap between general ISR research and game-specific applications raises key questions: How well do these models perform on dynamic, real-time game content? Can they run efficiently on lower-end GPUs? Which architectures best balance perceptual quality and performance for gaming?

We compare three model families: CNNs, GANs, and Transformers. Each team member developed one model using a curated gaming dataset from the Unity gaming engine. We built strong baselines, refined architectures, and evaluated perceptual quality to determine which approaches best suit video game upscaling.

## Background

ISR has advanced significantly through deep learning. Early research used shallow convolutional neural networks, while newer methods use adversarial training and perceptual loss for greater realism.

**SRCNN** (2014) pioneered deep learning for ISR with a three-layer architecture that outperformed interpolation. However, it struggled to capture complex features and relied on pre-upsampled inputs.

**VDSR** expanded to 20 layers and introduced residual learning to address vanishing gradients. It improved pixel-level accuracy but often produced overly smooth images.

**SRGAN** advanced the field by combining adversarial and perceptual losses, producing photorealistic textures. Yet, it sometimes introduced artifacts due to unstable training.

**ESRGAN** improved training stability and perceptual quality by adding dense residual blocks and expanding the generator architecture. However, its computational cost limits real time use in gaming.

**Transformers** now represent the cutting edge, using attention mechanisms to model long range dependencies. Though effective across many image types, they require high computational resources.

CNNs, GANs, and Transformers, offer different trade offs in accuracy, quality, and efficiency. Our project examines their performance specifically on video game imagery with its dynamic, synthetic, and stylized content.

## Methods

### CNNs

We implemented a Very Deep Super-Resolution (VDSR) model following the architecture proposed by Kim et al. (2016) for image super-resolution. The model takes low-resolution images, applies 18 convolutional layers with 64 filters of size  $3 \times 3$ , and uses ReLU activations. A total of 20 layers including output and input. To improve convergence speed, the model employs residual learning, where the residual image (difference between the high-resolution and low-resolution images) is predicted and added back to the input low-resolution image. This allows for faster learning of high-frequency details.

For data preparation, we utilized bicubic interpolation to scale the low-resolution images to the target resolution. The images were preprocessed by converting them to grayscale, and patches of size  $41 \times 41$  were extracted from the high-resolution and low-resolution images to create a dataset suitable for training. The patches were then used for training, validation, and testing with no overlap, ensuring that each patch was unique. The VDSR model was trained on a dataset of 200 images, and the model's performance was evaluated using the Peak Signal-to-Noise Ratio (PSNR) metric.

### GANs

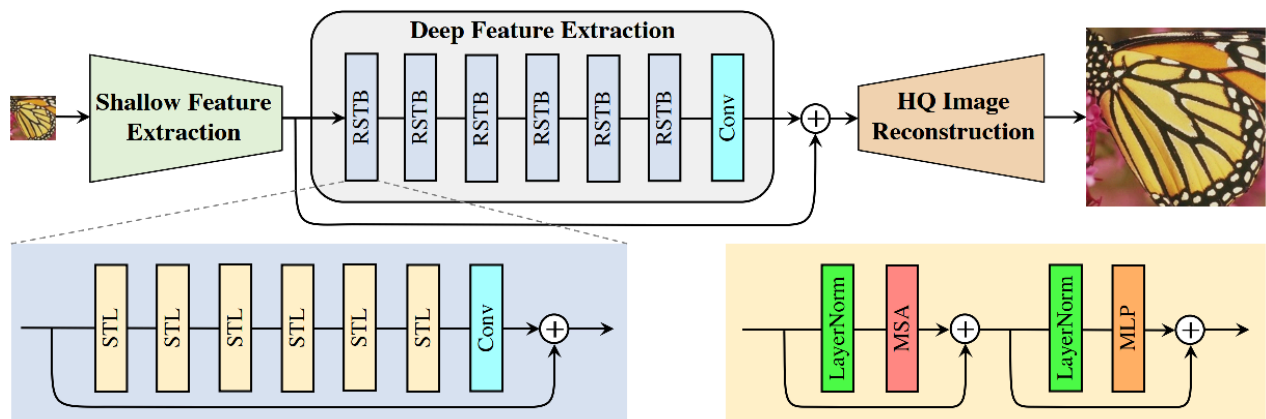
We implemented a Super Resolution GAN based model using a modified ESRGAN architecture. The generator received low-resolution images and passed them through a  $9 \times 9$  convolutional layer followed by a series of Residual-in-Residual Dense Blocks (RRDBs) without batch normalization to improve feature learning and reduce artifacts, following the modifications proposed in ESRGAN (Wang et al., 2018). We used a skip connection after the initial convolution and fused the output with the final feature maps before upsampling. Two  $4 \times$  upsampling blocks generated the high-resolution output.

The discriminator remained standard and evaluated both generated and real high-resolution images using a series of convolutional layers with Leaky ReLU activations. The model then output a probability score indicating whether the input was real or fake.

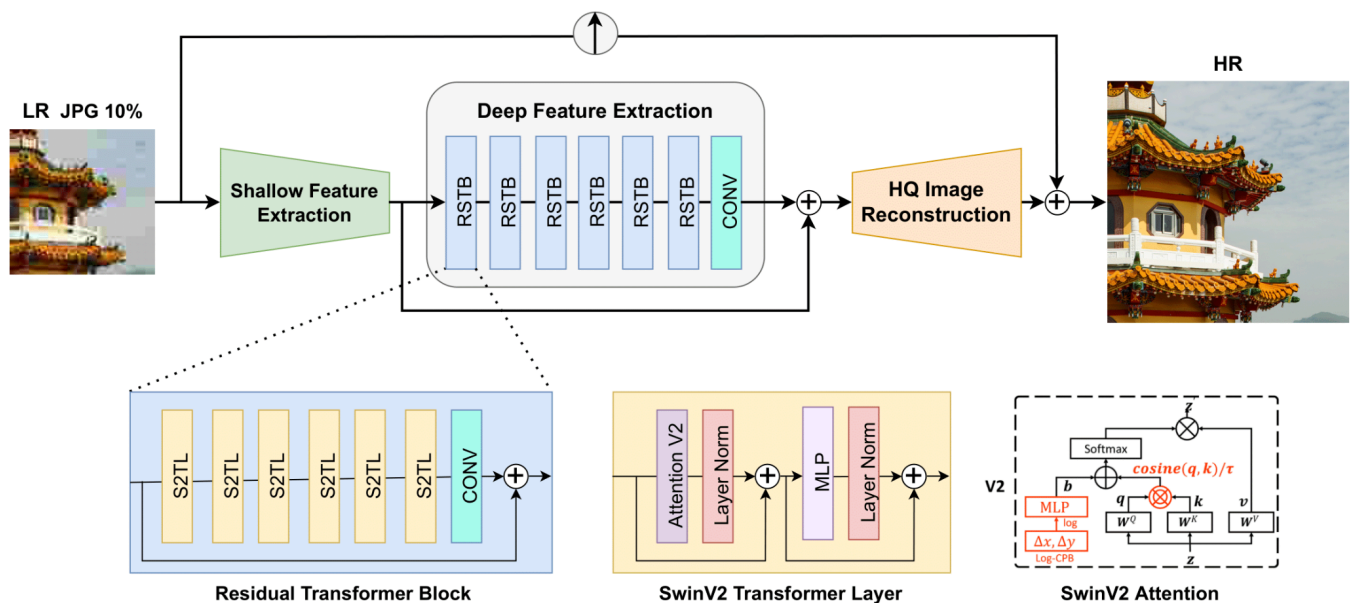
We combined three loss functions: L1 pixel loss, adversarial loss, and perceptual loss using VGG19 feature maps. This combination encouraged both visual realism and structural fidelity. We trained the SRGAN on a dataset of 14,000 paired low- and high-resolution images, optimizing with Adam (see Appendix for complete architecture diagrams).

## Transformers

We implemented a Super Resolution transformer based model using Swin2SR. This is an improvement upon the original SwinIR and its Swin transformer blocks. “Swin” is shorthand for shifted windows. Thus, SwinIR represents image resolution using shifted windows and Swin2SR represents super resolution using shifted windows. We use Swin2SR because it utilizes a newer version of the Swin transformer blocks, specifically to improve performance on the super resolution task (Conde et al., 2022).



The original SwinIR architecture consists of three main parts: shallow feature extraction, deep feature extraction, and HQ image reconstruction. Within the deep feature extraction lies 6 residual “swin” transformer blocks (RSTB) and a convolution layer. Within each of the RSTBs lies 6 “swin” transformer layers and a convolution layer. Of course, within the “swin” transformer layers is the attention mechanism.



The improved Swin2SR variant makes use of SwinV2 transformers instead of the original swin transformer layers. It utilizes post-normalization instead of pre-normalization and scaled cosine attention, which increased performance on super resolution tasks by 33% (Liu et al., 2021). The rest of the architecture remains the same.

## Experiments

### CNNs

#### Baseline

We trained the VDSR model for 80 epochs on a dataset of 200 low-resolution and high-resolution image pairs for training, validation, and testing. The model was trained using the NVIDIA A100 GPU on Google Colab. The VDSR model follows the architecture proposed by Kim et al. (2016), consisting of 18 convolutional layers, each with 64 filters, and using ReLU activations. Below are the main parameters

Patch Size: 41x41

Batch Size: 64

Number of Images: 200 LR~HR pairs

Learning Rate: 0.001 (decays every 20 epochs by a factor of 10)

Epochs: 80

Optimizer: Adam

Loss Function: Mean Squared Error (MSE)

Super Resolution for Video Games  
Eric Phann, Samantha Michael, Kidus Kidane  
[Github Link](#)

We evaluated model performance using Peak Signal-to-Noise Ratio (PSNR), which measures the ratio between the maximum possible power of an image and the power of corrupting noise that affects its quality. Higher PSNR values indicate better image reconstruction quality, with typical values for acceptable image quality ranging from 20 to 40 dB.

$$\text{PSNR} = 20 \cdot \log_{10} \left( \frac{\text{MAX}_I}{\sqrt{\text{MSE}}} \right)$$

Figure 1: Peak Signal to Noise Ratio

We also evaluated model performance using Structural Similarity Index Measure (SSIM), which predicts the perceived quality of an image, or its similarity to another image as a ratio from 0.0 to 1.0.

The training and validation graphs show that the model stopped at the 12th epoch due to early stopping, as the validation loss plateaued and no further significant improvements were observed. The PSNR increased quickly in the early epochs, but after reaching a peak, it stabilized, indicating that the model had reached its performance limit with the current configuration.

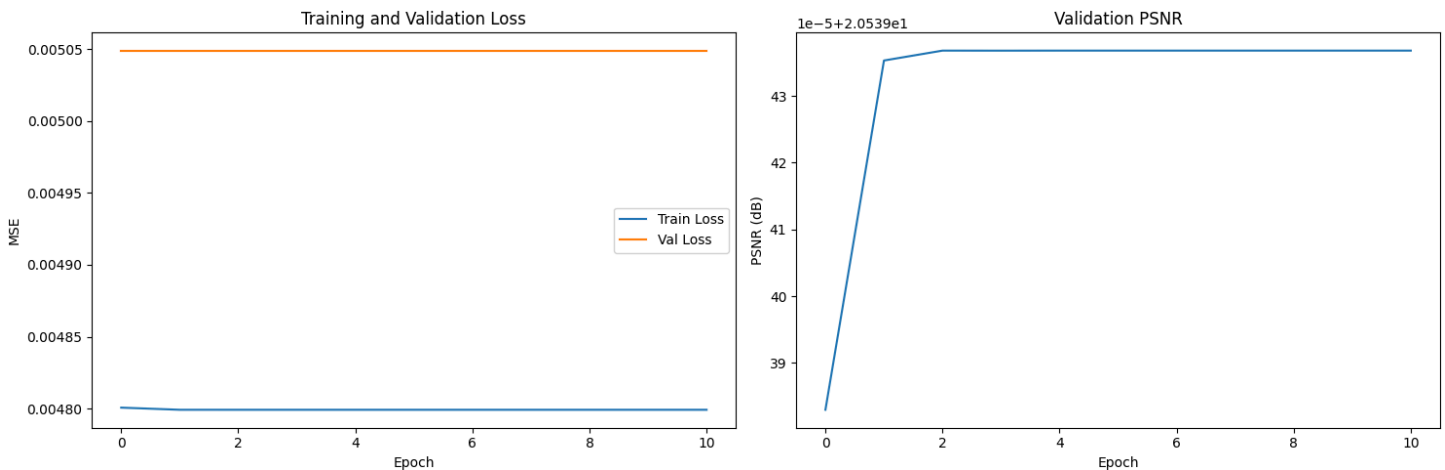


Figure 2: Training and Validation loss

The final test PSNR of **26.20 dB** is within the acceptable range, but there is still a noticeable difference in the structural quality of the image. While the output captures overall scene structure, edge artifacts and missing fine details, such as textures on the floor and furniture, are visible. The SSIM score of  $\sim 0.7574$  reflects moderate structural similarity, but textural accuracy remains an issue despite the high PSNR value

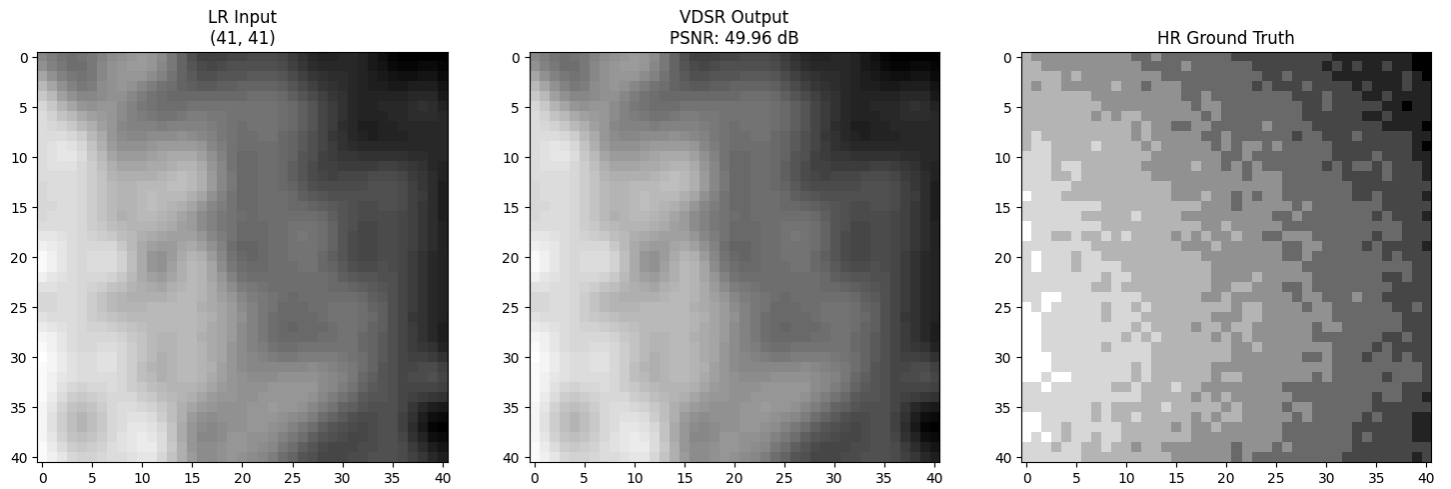


Figure 3: LR Input, VDSR Output, HR Ground Truth

## Model Tuning & Final Results

We used Optuna to optimize the VDSR model's hyperparameters, efficiently exploring values for model depth, filter size, and learning rate. The best configuration was:

**Depth:** 35 layers (input, 33 hidden layers, 1 output)

**Filters:** 64 filters

**Learning Rate:** 0.000202

The model achieved a test PSNR of **30.74 dB** and SSIM of **0.7574**, indicating good overall structure recovery but some loss in fine details. Early stopping occurred at epoch 8, suggesting the model reached its optimal performance early.

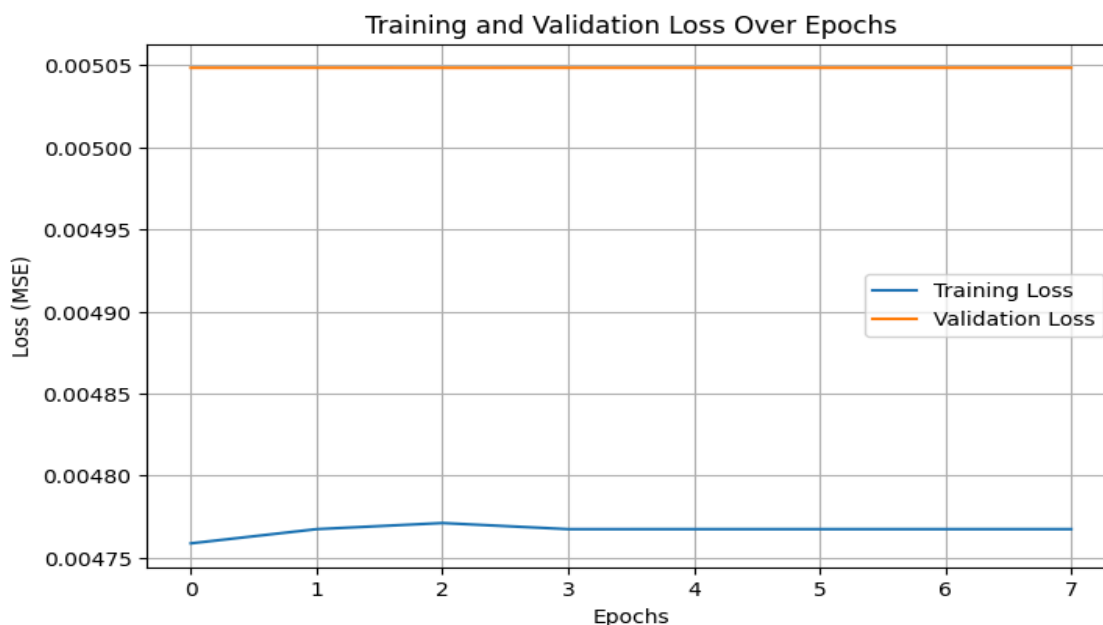


Figure 4: VDSR Tuned- Training and Validation loss

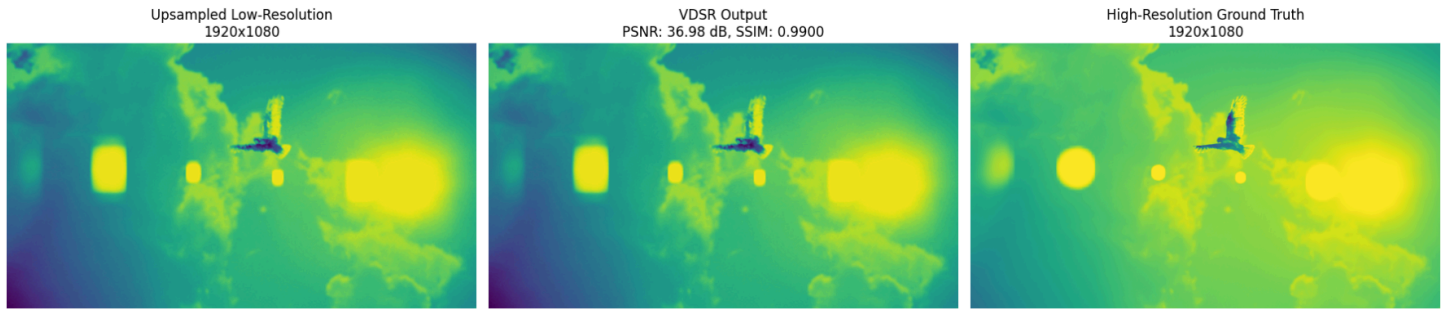


Figure 5: VDSR Tuned- LR Input, VDSR Output, HR Ground Truth

## Observations & Analysis

The baseline VDSR model demonstrated efficient learning, with the validation PSNR increasing rapidly in the early epochs and the MSE stayed consistent at a 0.005. This indicated that the model quickly captured the main structures and patterns of the images. The test PSNR of 26.20 dB, while within an acceptable range, highlighted issues, as the structural quality of the output still exhibited edge artifacts and missing fine details.

After fine-tuning the model with Optuna, significant improvements were achieved. The tuned model reached a higher test PSNR of **30.74 dB** and SSIM of **0.7574**, showing better structure recovery and clearer edges. Early stopping was applied at epoch 8, indicating that the model had reached its optimal performance earlier than expected.

Despite these improvements, the PSNR value, even at higher levels, may not fully reflect the quality of fine details, as demonstrated in the baseline model. This highlights that PSNR, while useful for measuring overall structure, does not always capture the recovery of intricate textures and finer details. The results show that while the model learned quickly and stabilized early, further improvements in capturing detailed textures are necessary for achieving higher-quality super-resolution.

## GANs

To evaluate SRGAN effectiveness for video game image super-resolution, we trained and tested two models: a baseline SRGAN and a tuned version using Optuna hyperparameter optimization comparing their performance on the Super Resolution in Video Games dataset (Kaggle, n.d.).

### Baseline

We trained the baseline model for 500 epochs on a small subset of the dataset, consisting of 50 low-resolution and high-resolution image pairs each for training, validation, and testing, generated from the Unity engine. The baseline configuration was performed on Google Colab using a NVIDIA A100 GPU. The configuration used no custom weight initialization and had the following parameters:



Generator learning rate: 1e-3

Discriminator learning rate: 5e-5

Batch size: 4

Number of residual blocks: 24

LeakyReLU alpha: 0.2

Generator:Discriminator update ratio: 2:1

Patch sizes: 64px (LR) → 256px (HR)

The model achieved a PSNR of 17.92 dB (below the acceptable 20-40 dB range). As Figure 2 shows, while the output captures overall scene structure and color balance, it exhibits noticeable edge artifacts around windows and furniture. Fine details are lost in floor patterns, wall decorations, and plant textures, confirming the model's difficulty reconstructing structural and textural details despite capturing basic scene elements.

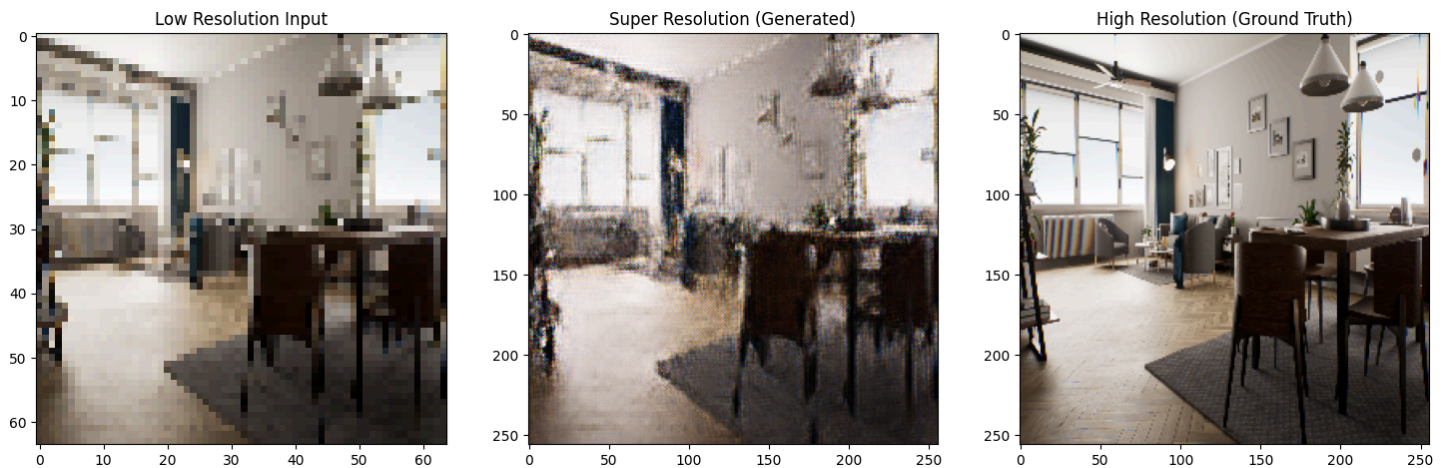


Figure 6: Baseline GAN Output

## Model Tuning & Final Results

We ran 10 Optuna trials to optimize key hyperparameters, including the number of residual blocks, filter size, dropout rate, and learning rates. A summary of the full set of tuned parameters is provided in the Appendix.

After identifying the optimal hyperparameters, we trained the final model for 200 epochs on the full dataset using the university's HPC cluster, equipped with a NVIDIA TITAN RTX GPU. This setup enabled efficient large-scale training on the full dataset and ensured stable convergence under the tuned configuration.

This model achieved a PSNR of 19.70 dB, a ~1.8 dB improvement over the baseline. We also applied color correction post-processing, which enhanced visual fidelity and better matched the target high-resolution images.



Example 4

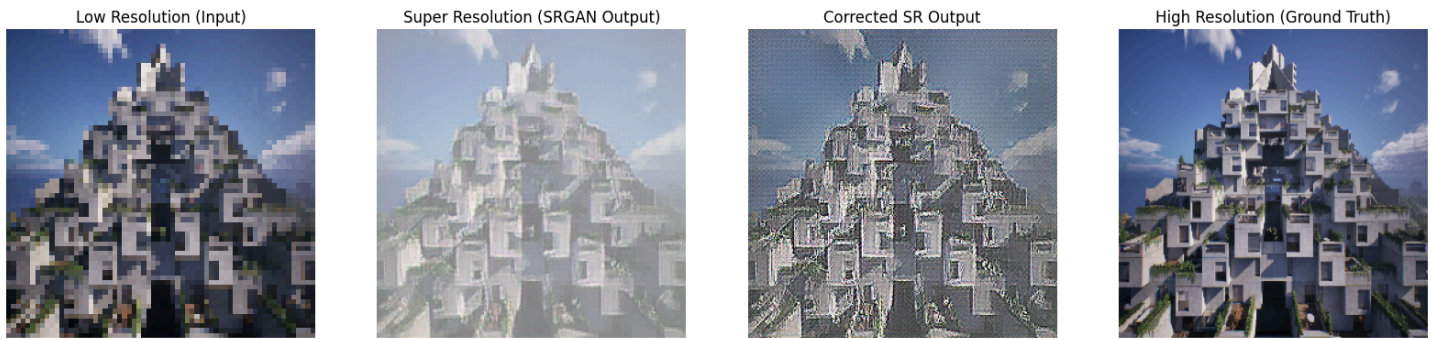


Figure 7: Tuned GAN Output

## Observations & Analysis

Training stability improved significantly after applying weight initialization and refining key hyperparameters. The tuned model demonstrated smoother convergence and reduced noise across epochs.

Qualitatively, the final outputs showed sharper edges, improved structure preservation, and better color depth, particularly in high-frequency areas such as furniture edges, wall textures, and lighting transitions. While the final PSNR remains below state-of-the-art SR benchmarks (25-30 dB), the tuned SRGAN achieved meaningful improvements with a relatively compact architecture, making it suitable for real-world scenarios with resource constraints.

These results suggest that even modest architectural tuning and optimization can substantially improve visual and structural fidelity in GAN-based super-resolution tasks.

## Limitations & Next Steps

Future experiments may involve adopting deeper architectures such as ESRGAN by increasing the number of RRDBs. Implementing early stopping to prevent overfitting, or incorporating perceptual metrics beyond PSNR. Additionally, expanding the hyperparameter search space with more Optuna trials would enable finer grained tuning and potentially yield further performance gains.

## Transformers

### Baseline

For our baseline approach of transformers for super image resolution, we used a pre-trained Swin2SR model available via Hugging Face. For more details on model architecture please refer to the Methods section or the appendix. The Hugging Face API makes it easy to load in pretrained models in just a few lines of code:

```
1) !pip install -q git+https://github.com/huggingface/transformers.git
```

```
2) from transformers import Swin2SRForImageSuperResolution
3) model =
    Swin2SRForImageSuperResolution.from_pretrained("caidas/swin2SR-classical-sr-x4-64")
```

This model is the model introduced in the Swin2SR paper by Conde et al. and upscales images by x4. While the model was easy to implement, due to the nature of our data being images, we needed to preprocess the images before it would work with the model. Thankfully, due to the Hugging Face API, we are able to utilize a processor specifically for the Swin2SR model we are using, which does a lot of the heavy lifting. Aside from that, we only needed to adjust the pixel values array shape to view it properly.

```
from transformers import AutoImageProcessor

processor = AutoImageProcessor.from_pretrained("caidas/swin2SR-classical-sr-x4-64")
```

After testing that our baseline model would successfully output a high resolution image prediction, we can then test it using a low resolution example. We took the model's high resolution prediction and compared it against our "ground truth" using our two metrics, PSNR and SSIM.



With this one example, our model produced a PSNR of **17.85 db** and an SSIM of **0.4825**. Although neither metric is as high as we'd like, the model is working. We will look at potential improvements in the next section.

## Model Tuning & Final Results

For our model improvement we decided to fine-tune the pre-trained Swin2SR model on some of our video game dataset. The intuition behind this is that the model will be able to perform better on this unique domain.

Although we wanted to leverage the native Hugging Face library for training the model, the researchers had not implemented the training loop accordingly. Thus, we needed to build our own training loop from scratch using PyTorch.

We froze all the model's parameters, then unfroze the upsample and the last couple layers of the model. Without freezing most of the model parameters, the Colab notebook would run out of memory and crash.

Additionally, the intuition behind only unfreezing the last couple of layers is that we assume the model has sufficiently pretrained on the general features and only needs tuning to our specific domain of video games.

The parameters we used are as follows. The AdamW optimizer was selected due to efficiency in directly applying weight decay as opposed to the gradients. Charbonnier was selected as the loss function as a balance between L1 and L2 loss. Lastly we froze all layers except the last two and the subsampling layer.

We then trained 250 low-res/high-res image pairs over 5 epochs, taking about 1 hour to complete. The final results, averaged across, are as follows:

PSNR (Base-line): 25.05 dB vs. PSNR (Fine-tuned): **24.11 dB**

SSIM (Base-line): 0.7671 vs. SSIM (Fine-tuned): **0.7658**

## Observations & Analysis



It is important to note that although our fine-tuned model performed quantitatively poorer than the base-line pre-trained model, that *perceived* visual quality may still increase. These two metrics are known to be somewhat misleading, especially for the super resolution task. For example, in the image above, the fine-tuned model actually learns the edges of the sky line better than the base-line model, despite the fine-tuned model on average performing poorer.



## Limitations & Next Steps

A limitation of this approach is that there is not that much customizability available when compared to the other models. For this approach, we wanted to look at state of the art transformer models, so it is difficult to innovate upon and improve upon research quality work efforts.

A potential next step is to fine-tune this model on a cluster. After perhaps training on the entire dataset or for more epochs, we might see the PSNR and SSIM metrics noticeably increase as well as visually see that the model is improving upon the video game domain.

## Conclusions

We successfully reproduced and adapted a GAN-based super-resolution model using a modified ESRGAN architecture. Through baseline evaluation and targeted hyperparameter tuning, we improved performance from a PSNR of 17.92 dB to 19.70 dB, with clear gains in edge sharpness, structural preservation, and visual realism.

A key learning from this work was the challenge of transitioning from prototyping in Colab to full-scale training on the HPC cluster. Scaling up to 14,000 image pairs required substantial code modifications to improve batch handling, data loading pipelines, and memory management, underscoring the importance of engineering effort in deep learning research.

While state-of-the-art PSNR benchmarks were not reached, the results demonstrate the effectiveness of architectural tuning and optimization when applied to real-world datasets under constrained resources.

For CNN implementation, we successfully implemented and fine-tuned the VDSR model for super-resolution tasks. Through hyperparameter optimization using Optuna, we improved the model's performance from a PSNR of 26.20 dB to 30.74 dB, with improvements in edge sharpness and structural preservation. While PSNR proved to be a valuable metric, it didn't fully capture the finer details, particularly textures and small features. Bicubic interpolation likely helped the model learn quickly by providing better quality low-resolution images for training. The fine-tuned model also performed well on unseen data, demonstrating its generalization capabilities with a respectable PSNR of **30.74 dB**

Finally, the transformer approach allowed us to take a look at the next frontier of image super resolution by leveraging state-of-the-art architectures and fine-tuning it for specific use in the video game domain.

## Response to Feedback

Based on feedback from the instructor, we implemented:

**Loss Visualization:** Generator and discriminator losses were plotted on separate charts to account for their differing scales. This adjustment made the training dynamics easier to interpret and highlighted stability improvements in the tuned model. (See Appendix)

**Full Architecture Diagram:** A complete diagram of the model architecture was created to illustrate the end-to-end structure of the generator and discriminator components. This visual aids in understanding the data flow and key architectural elements. (See Appendix)

**Code Documentation and Citations:** Relevant portions of the code were updated with citations and links to source material. In particular, references to the ESRGAN architecture and its components.

These refinements improved both the transparency and reproducibility of the project, while also aligning the final deliverable more closely with best practices in research and engineering.

## References

Conde, M. V., Choi, U.-J., Burchi, M., & Timofte, R. (2022, September 22). *Swin2SR: SwinV2 transformer for compressed image super-resolution and restoration*. arXiv.org. <https://arxiv.org/abs/2209.11345>

Dong, C., Loy, C. C., He, K., & Tang, X. (2014). *Image super-resolution using deep convolutional networks*. arXiv. <https://arxiv.org/abs/1501.00092>

Kim, J., Lee, J. K., & Lee, K. M. (2016). *Accurate image super-resolution using very deep convolutional networks*. arXiv. <https://arxiv.org/abs/1511.04587>

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021, August 17). *Swin Transformer: Hierarchical vision transformer using shifted windows*. arXiv.org. <https://arxiv.org/abs/2103.14030>

Simonyan, K., & Zisserman, A. (2014). *Very deep convolutional networks for large-scale image recognition*. arXiv. <https://arxiv.org/abs/1409.1556>

Wang, X., Yu, K., Wu, S., Gu, J., Liu, Y., Dong, C., Qiao, Y., & Change Loy, C. (2018). *ESRGAN: Enhanced super-resolution generative adversarial networks*. arXiv. <https://arxiv.org/abs/1809.00219>

## Datasets

Super Resolution in Video Games (Kaggle Competition Dataset)

Kaggle. (n.d.). Super resolution in video games [Data set].

<https://www.kaggle.com/competitions/super-resolution-in-video-games/data>

<https://huggingface.co/datasets/ericphann/video-game-super-resolution>

## Appendix

[Video Game Super Resolution Github](#)

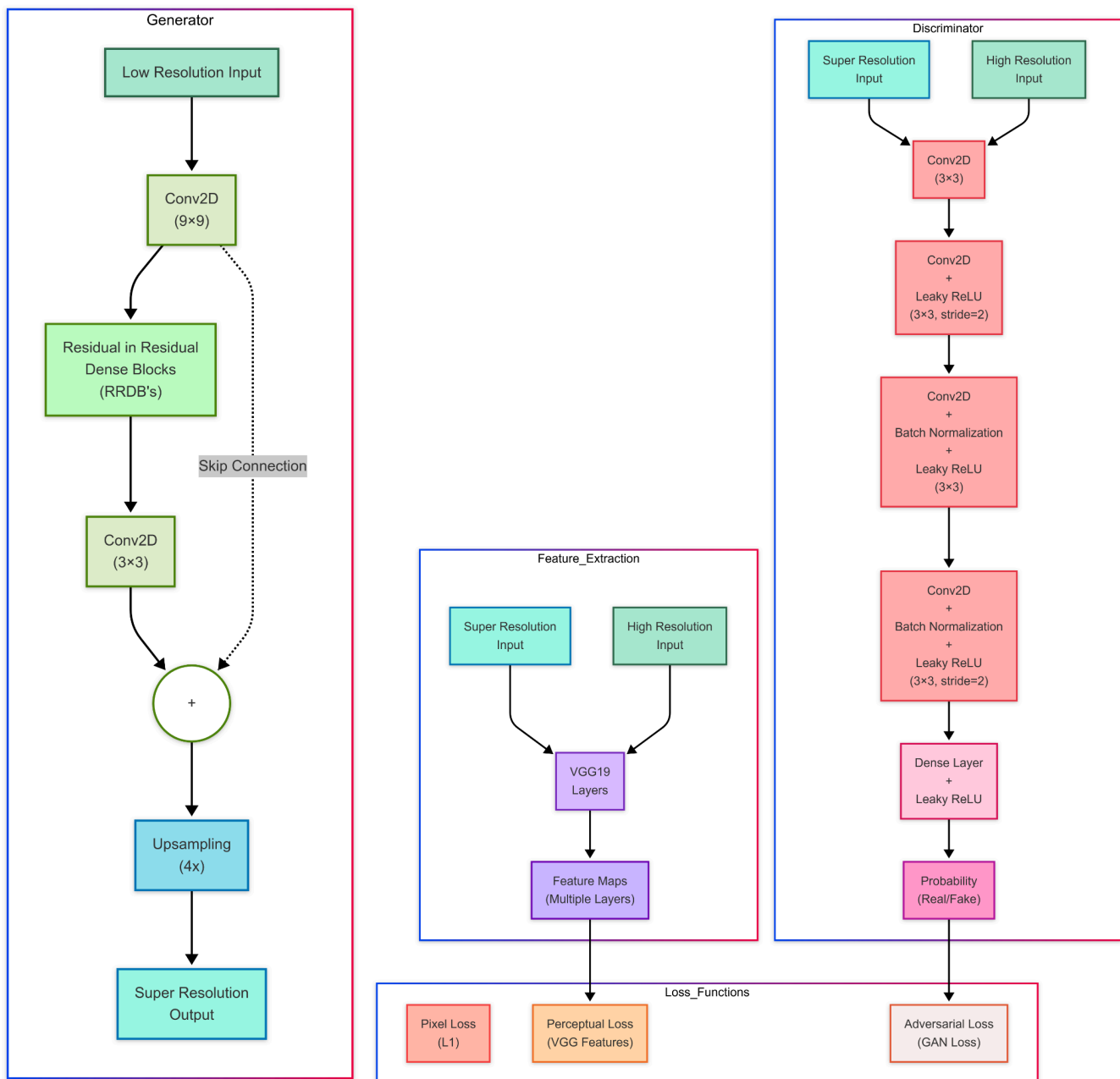


Figure A.1: GAN Architecture Diagram

Super Resolution for Video Games  
Eric Phann, Samantha Michael, Kidus Kidane  
[Github Link](#)

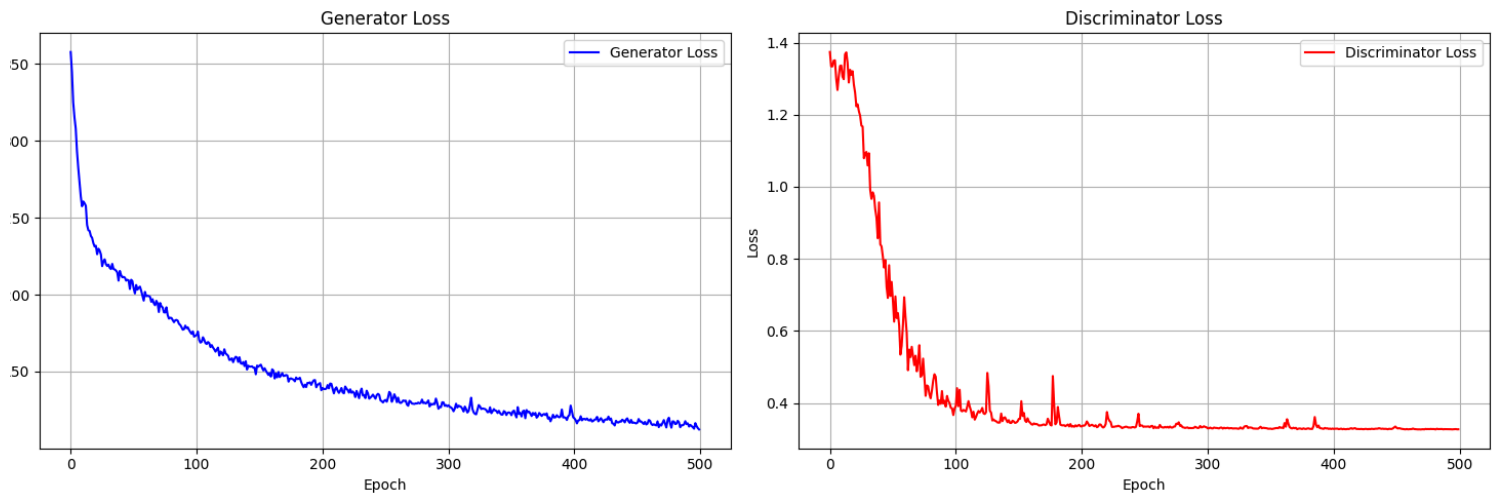


Figure A.2: Baseline generator and discriminator loss curves plotted separately to reflect their differing scales.

| Best Trial #7 Results   |                        |
|-------------------------|------------------------|
| PSNR: 19.69608497619629 |                        |
| Hyperparameters         |                        |
| Parameter               | Value                  |
| num_residual_blocks     | 8                      |
| filters                 | 96                     |
| dropout_rate            | 0.2971465273570587     |
| lr_gen                  | 4.6750250331130036e-05 |
| lr_disc                 | 1.1942012783540352e-05 |
| perceptual_loss_weight  | 0.028146790912123167   |
| pixel_loss_weight       | 0.1481637014313447     |
| adv_loss_weight         | 0.07625602618459658    |

Table A.1: Best-performing GAN hyperparameters identified through Optuna tuning (Trial #7)



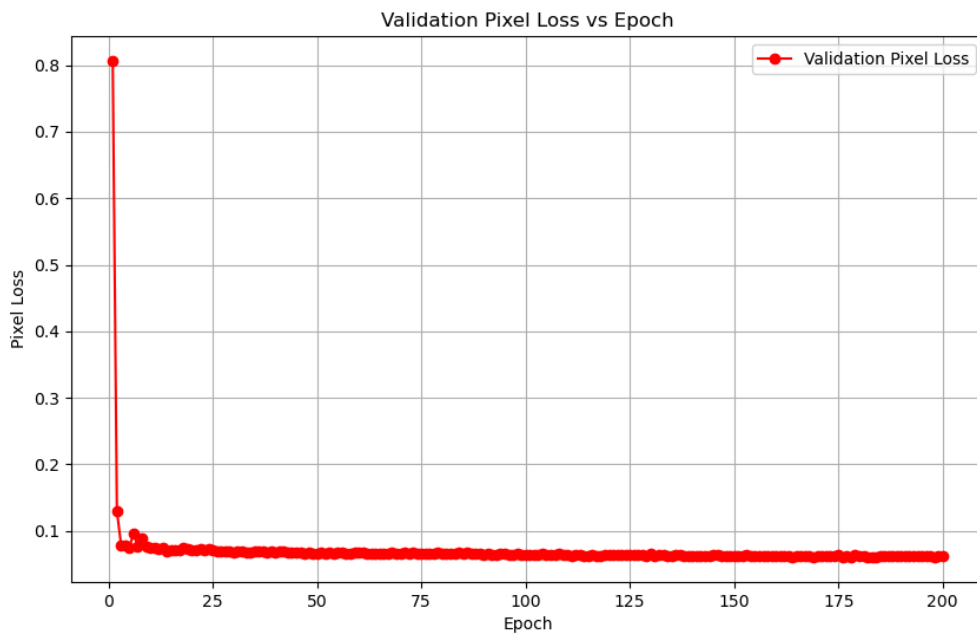


Figure A.3: Validation performance of the tuned SRGAN model over 200 epochs.

Validation pixel loss rapidly decreases and stabilizes after the first few epochs, indicating improved convergence compared to the baseline.

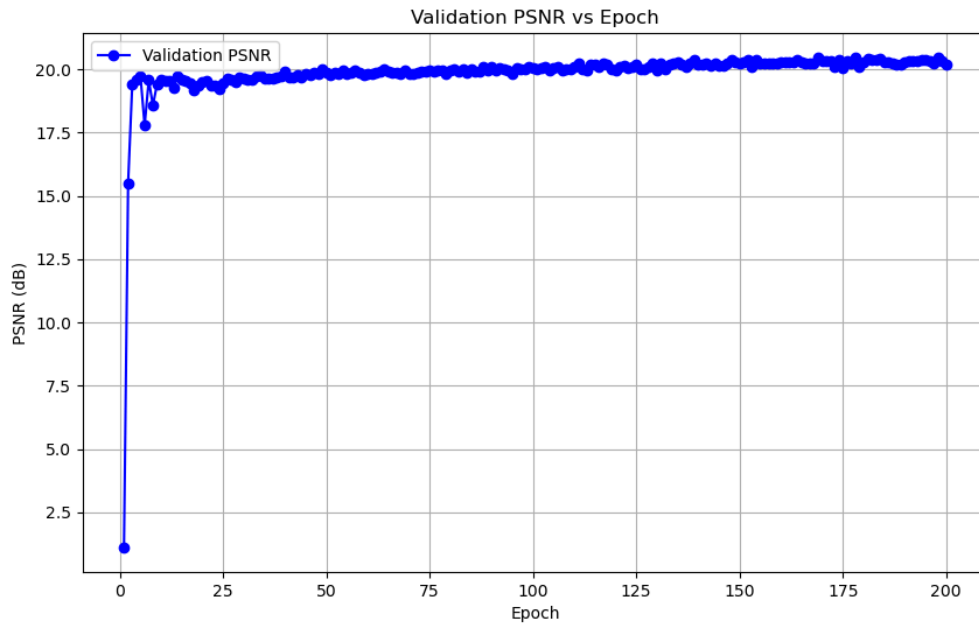


Figure A.4: Tuned SRGAN PSNR improves consistently and plateaus around 19.7 dB, demonstrating overall reconstruction stability and perceptual quality improvement across epochs.

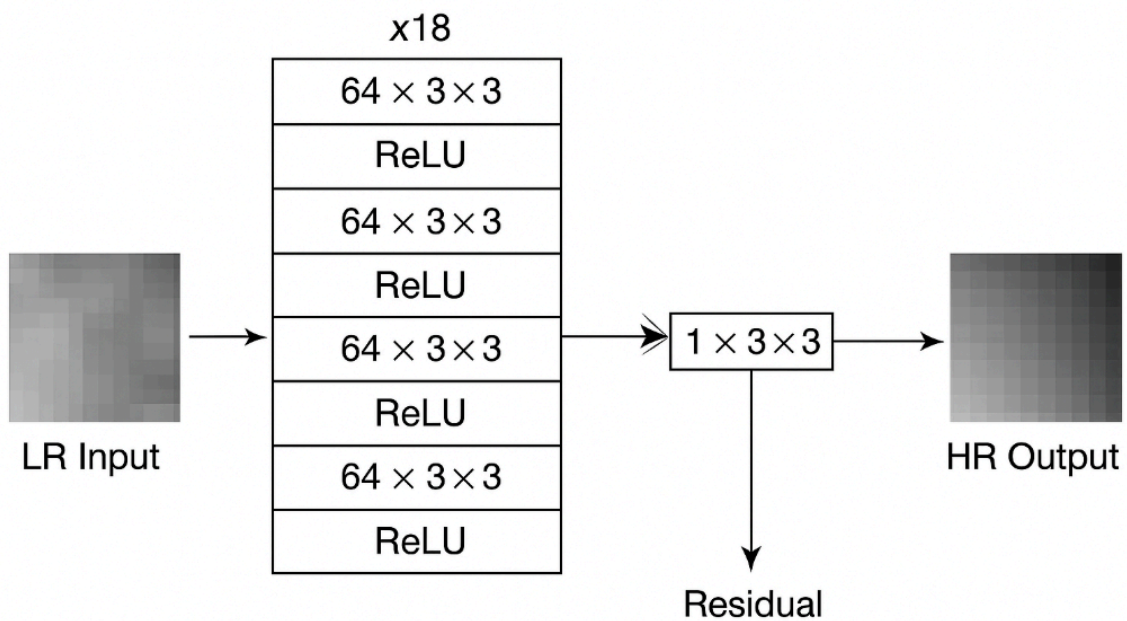


Figure A.5: VDSR Architecture